

Taxímetro Digital

TaxiTech Solutions — Sistema de facturación de flota

Memoria del proyecto

Anna Costa

Factoria F5 — Bootcamp Fullstack, módulo M1

Repositorio: github.com/costanna/project-py-taximetro

Demo en vivo: taxy-py.vercel.app

Septiembre 2026

1. Resumen ejecutivo

TaxiTech Solutions es una empresa de gestión de flotas de taxi que ha decidido sustituir sus taxímetros físicos, sin soporte del fabricante desde 2023, por un sistema 100% software. Este proyecto desarrolla el prototipo piloto encargado por el CTO de la compañía: un taxímetro digital que calcula la tarifa en tiempo real según el estado del vehículo (parado o en movimiento), permite encadenar carreras sin reiniciar el sistema, deja constancia del histórico para cuadrar caja y protege el acceso con usuario y contraseña.

El desarrollo se ha organizado en las cuatro fases incrementales definidas por el cliente (MVP funcional, observabilidad y persistencia, arquitectura y experiencia de usuario, y versión de producción), cubriendo las nueve historias de usuario priorizadas por MoSCoW. El resultado final está desplegado y accesible en producción: una API REST (Render + PostgreSQL) y una interfaz web (Vercel), con **104 tests automatizados** y un pipeline de integración continua que además comprueba el despliegue con Docker.

2. Contexto y encargo

El equipo de operaciones de TaxiTech Solutions lleva desde 2018 usando taxímetros físicos Hale T200, sin soporte desde 2023. La dirección quiere migrar a un sistema software antes de Q3 2025, y ha abierto este proyecto piloto para validar el concepto antes de comprometer presupuesto con un proveedor externo.

Briefing del cliente (Director de Operaciones):

«Necesitamos algo que los taxistas puedan arrancar al inicio del turno y que calcule lo que le cuesta al pasajero en tiempo real. Cuando el taxi está parado en un semáforo, el contador sigue corriendo pero más despacio. En marcha, corre más rápido. Al llegar al destino, el taxista pulsa un botón, sale el total, y listo. Nos gustaría también poder ver un histórico de carreras del día. Si puede tener contraseña para que no lo toquen los pasajeros, mejor. Y si en el futuro se puede ver desde el móvil o una tablet en el taxi, perfecto.»

Tarifas vigentes (Zona EMT Madrid, junio 2025):

- Taxi parado o velocidad < 20 km/h: 0,02 €/segundo
- Taxi en movimiento: 0,05 €/segundo

3. Historias de usuario

Priorizadas por el cliente con MoSCoW:

ID	Historia	Prioridad
US-01	Iniciar una carrera con un solo comando	Must
US-02	Cambiar entre parado y en movimiento, ajustando la tarifa	Must
US-03	Finalizar la carrera y ver el total en euros	Must

ID	Historia	Prioridad
US-04	Encadenar carreras sin cerrar el programa	Must
US-05	Ver el histórico de carreras del día para cuadrar caja	Should
US-06	Generar logs de operación para diagnosticar errores	Should
US-07	Cambiar las tarifas sin redeployar	Should
US-08	Requerir contraseña para proteger el sistema	Could
US-09	Interfaz visual con botones grandes para móvil/tablet	Could

Las nueve historias están implementadas y verificadas con tests automatizados. La US-05 incorpora además un matiz no contemplado inicialmente: distinguir entre el rol de *taxista* (ve solo sus propias carreras) y el de *responsable de flota* (ve el histórico de todos los conductores, con el total de cada uno y el total general), necesario en cuanto el sistema pasó a admitir varios usuarios. El historial muestra, para cada carrera, el conductor, la hora de inicio, la duración y el importe, junto con lo recaudado en el día.

4. Arquitectura

4.1. Dos implementaciones

El repositorio contiene dos implementaciones completas del mismo dominio, construidas en momentos distintos del proyecto:

- **taximetro/ + web/** — la referencia usada para desarrollar y probar las cuatro fases: CLI y API en Flask, persistencia en SQLite, pensada para self-host con Docker (*docker compose up*, un único comando).
- **backend/ + frontend/** — la que se despliega en producción: API en FastAPI con base de datos PostgreSQL (Neon) y una interfaz web estática servida desde Vercel.

Ambas comparten la misma lógica de negocio (acumulación de tarifa por tramos, gestión de estados, autenticación con contraseña hasheada) y las mismas historias de usuario, verificadas cada una con su propia batería de tests. La diferencia de alcance es que la referencia Flask modela un único taxi por instancia, mientras que el backend de producción lleva una carrera independiente por conductor.

4.2. Modelo de datos (backend de producción)

Entidad	Campos relevantes	Notas
Carrera	id, usuario, estado, importe_acumulado, en_curso, inicio, fin, ultimo_cambio	Una fila por carrera, en curso o finalizada, ligada al conductor que la realiza
Usuario	id, username, password_hash, rol	password_hash con scrypt (werkzeug); rol: taxista responsable
Tarifas	id, tarifa_parado, tarifa_movimiento	Fila única, editable en caliente desde el panel del responsable, sin redeploy

El importe se calcula siempre en el servidor a partir de las marcas de tiempo: cada cambio de estado cierra el tramo anterior con su tarifa y abre uno nuevo. El navegador solo muestra el resultado, de modo que recargar la página o perder la conexión no altera lo que se cobra.

4.3. Autenticación y roles

El acceso requiere usuario y contraseña (US-08). Las contraseñas nunca se guardan en texto plano: se derivan con `scrypt` y `sal` aleatoria (`werkzeug.security`). El registro exige un usuario de 3 a 30 caracteres y una contraseña de al menos 8. Las sesiones usan tokens firmados con caducidad de 8 horas (un turno), sin necesidad de una tabla de sesiones.

El primer usuario que se registra en cada instalación se marca automáticamente como **responsable de flota**; el resto son **taxistas**. El rol determina qué carreras se ven en el histórico: un taxista solo las suyas, un responsable todas, con el desglose por conductor y el total general. El responsable puede además cambiar las tarifas y finalizar una carrera que otro conductor haya dejado abierta.

4.4. Reglas de negocio protegidas

- Un conductor no puede tener dos carreras en curso a la vez: un segundo inicio se rechaza y la interfaz retoma la carrera existente.
- Cambiar las tarifas no recalcula lo ya recorrido: el tramo en curso se cierra con la tarifa anterior y la nueva se aplica desde ese instante.
- Un taxista no puede ver ni modificar la carrera de otro.
- Al cerrar sesión con una carrera en curso, la interfaz avisa de que sigue contando y ofrece finalizarla; al cerrar la pestaña, el navegador pide confirmación.

5. Desarrollo por fases

Fase	Historias	Entregable
1 — MVP funcional	US-01 a US-04	CLI operativo con el flujo completo de una carrera
2 — Observabilidad y persistencia	US-05, US-06, US-07	Logging estructurado, historial persistente, tarifas configurables
3 — Arquitectura y UX	US-08, US-09	Refactor a POO por responsabilidades, autenticación, interfaz táctil
4 — Producción	—	Base de datos relacional, API REST, panel web, despliegue en un comando

Cada fase se implementó como un módulo autocontenido dentro del paquete *taximetro/* (core, logger, storage, config, auth, cli, api), reutilizando la lógica de negocio entre el CLI y la API en vez de duplicarla.

6. Calidad y pruebas

El proyecto cuenta con **104 tests automatizados** (64 sobre la referencia Flask y 40 sobre el backend de producción), distribuidos en tres niveles:

- **Unitarios** — lógica de tarifas con un reloj inyectado (sin depender de `time.sleep real`), carga de configuración, hashing y validación de tokens.
- **Integración** — rutas de la API completas (autenticación, ciclo de vida de una carrera, aislamiento de historial por usuario, cambio de tarifas a mitad de carrera) contra una base de datos de prueba real.
- **End-to-end** — un servidor HTTP real levantado en un hilo, recorriendo el viaje completo de un turno de taxi tal como lo haría un navegador.

El pipeline de integración continua (GitHub Actions) ejecuta linting (black, isort, flake8) y la suite completa de tests en cada push y pull request, en Python 3.11 y 3.12. Un tercer job levanta la aplicación con *docker compose up* y comprueba que la web responde, de modo que el despliegue con un único comando queda verificado en cada cambio. Un hook de pre-commit aplica el mismo formato antes de cada commit.

La interfaz web no tiene tests automatizados: se ha verificado manualmente y con capturas de un navegador sin interfaz. Es la principal carencia de la batería de pruebas.

7. Gestión del proyecto

El trabajo se gestiona en un tablero de GitHub Projects enlazado al repositorio, con una columna por fase del roadmap (Backlog, Fase 1 a Fase 4, En revisión y Done) y una tarjeta por cada historia de usuario y por cada requisito de la Fase 4, etiquetada con su prioridad MoSCoW.

Los cambios entran en *main* mediante pull requests desde ramas de trabajo (*feature/*, *fix/*, *chore/*, *docs/*), con mensajes en formato Conventional Commits y la CI ejecutándose en cada una. El flujo completo está descrito en CONTRIBUTING.md.

8. Despliegue

Componente	Dónde	Cómo
Referencia (taxímetro/ + web/)	Self-host / Docker	<code>docker compose up --build</code> ; servida con gunicorn
API de producción	Render (plan gratuito)	FastAPI + PostgreSQL (Neon)
Interfaz de producción	Vercel	HTML/CSS/JS estático
Base de datos	Neon (PostgreSQL gestionado)	Persistente, con roles y tarifas

Render y Vercel despliegan automáticamente cada vez que se fusiona un cambio en *main*. El despliegue en producción usa cuentas personales de la autora, separadas del repositorio de la organización del bootcamp. El repositorio de producción (github.com/costanna/project-py-taximetro) es independiente — no un fork — para poder gestionar su visibilidad y su conexión con Render y

Vercel sin restricciones.

Al estar en el plan gratuito de Render, el servicio entra en reposo tras un periodo de inactividad; la primera petición tras el reposo tarda unos segundos en responder ("cold start"). La interfaz lo avisa mientras espera. El repositorio incluye un workflow programado que hace ping a la API en horario de uso para reducir esos arranques, aunque GitHub no garantiza la puntualidad de las ejecuciones programadas, así que conviene abrir la aplicación un par de minutos antes de una demo en vivo.

9. Decisiones técnicas relevantes

Decisión	Motivo
SQLite en la referencia, PostgreSQL en producción	SQLite no requiere infraestructura para un prototipo de este tamaño; Postgres (Neon) da concurrencia real y persistencia gestionada en la nube
Tarifas en base de datos en producción; en config.json en la referencia	El plan gratuito de Render no tiene disco persistente ni acceso a Shell; guardarlas en la base de datos permite cambiarlas en caliente sin redeploy, con un endpoint protegido solo para el responsable de flota. En la referencia, el panel escribe el fichero de configuración que pide la US-07
Importe calculado en el servidor	El cobro no depende de que el navegador siga abierto ni de su reloj; la contrapartida es que una carrera olvidada sigue contando, y por eso la interfaz avisa al salir
Tokens firmados en vez de sesiones en base de datos	Evita una tabla y una consulta extra por petición; la caducidad de 8 h cubre un turno de trabajo completo
Versiones de dependencias acotadas	Una versión menor nueva de SQLAlchemy cambió el driver de Postgres por defecto y rompió un despliegue; se limitó a la serie 2.0.x
Repositorio de producción independiente del fork del bootcamp	GitHub no permite cambiar la visibilidad de un fork; duplicar el repositorio permite controlarla y conectarlo a Render/Vercel sin líos

10. Dificultades encontradas

- Un merge entre dos líneas de trabajo independientes (la referencia en Flask/SQLite y la versión de producción en FastAPI/Postgres) se quedó a medio resolver, con marcadores de conflicto de Git comiteados y subidos sin querer; hubo que detectarlo y arreglarlo antes de seguir.
- Una carrera de prueba quedó abierta casi 24 horas y acumuló 1.691 €, porque cerrar la pestaña no la detiene. De ahí salieron los avisos al cerrar sesión y al cerrar la pestaña, la opción de que el responsable cierre carreras ajenas y la regla de una sola carrera en curso por conductor.
- Un despliegue en Render falló sin haber cambiado el código afectado: el fichero de dependencias admitía cualquier SQLAlchemy 2.x y la versión 2.1 pasó a buscar un driver de Postgres distinto del instalado.
- El aviso de "servidor arrancando" no desaparecía en algunos navegadores porque un bloqueador de anuncios cortaba la ruta `/health`; ahora se comprueba otra ruta y cualquier respuesta de la API cuenta como señal de que está lista.
- Distintos navegadores (Edge, Brave) interfieren de forma distinta con los campos de contraseña personalizados, y el atributo `hidden` no oculta iconos SVG; la solución robusta fue simplificar el control a un botón de texto.
- El aislamiento de datos por usuario, necesario por privacidad entre conductores, entraba en tensión con la necesidad del responsable de flota de ver el conjunto — resuelto con un sistema de roles.

11. Líneas futuras

- Tests automatizados de la interfaz web, hoy verificada a mano.
- Panel de administración para gestionar altas de usuarios y cambios de rol sin acceso directo a la base de datos; hoy el registro está abierto a cualquiera.
- Límite de intentos de inicio de sesión y restricción de CORS al dominio de la interfaz.
- Migraciones de esquema versionadas (Alembic) en vez de alterar la base de datos de producción a mano.
- Cierre automático de las carreras que superen una duración máxima.
- Multi-vehículo: un Taxímetro por sesión/dispositivo en vez de una única instancia compartida en la referencia Flask.

12. Enlaces

Recurso	URL
Repositorio	github.com/costanna/project-py-taximetro
Aplicación en vivo	taxy-py.vercel.app
API en vivo (health check)	taximetro-api.onrender.com/health
Tablero Kanban	github.com/users/costanna/projects/3